

Statistiques et incertitudes

Décrire une série, valider une régression
et simuler la variabilité par Monte-Carlo

OBJECTIF

Exploiter une série de mesures, comparer des valeurs, ajuster un modèle affine et propager la variabilité par simulation.

Un calcul statistique ne remplace ni l'examen des données ni le raisonnement expérimental. Chaque résultat doit être associé à son modèle, ses unités et ses hypothèses.

LES SEPT SCRIPTS

Script	Notion	Usage
01_statistiques_type_a.py	Moyenne, écart-type et u sur la moyenne	Résultats d'une classe
02_ecart_normalise.py	Comparaison quantitative de deux valeurs	Résultat et référence
03_regression_lineaire.py	Régression avec <code>numpy.polyfit</code>	TP2 et TP6
04_valider_regression.py	Barres et résidus normalisés	Validation d'un étalonnage
05_tirages_normaux.py	Tirages avec <code>numpy.random.normal</code>	Introduction à la simulation
06_monte_carlo_grandeur_composee.py	Propagation d'un calcul de titrage	TP1, TP7, TP8, TP9, TP14
07_monte_carlo_regression.py	Incertitude sur la pente et l'origine	TP2 et TP6

Progression conseillée

Commencer par les scripts 01 et 02. Étudier ensuite la régression avec 03 et 04. Utiliser enfin 05 comme préparation aux simulations 06 et 07.

1. Décrire et comparer des résultats

Statistiques de type A

```
nombre = len(mesures)
moyenne = np.mean(mesures)
ecart_type = np.std(mesures, ddof=1)
u_moyenne = ecart_type / np.sqrt(nombre)
```

Grandeur	Question traitée
Moyenne	Quelle valeur retient-on pour la série ?
Écart-type expérimental s	Quelle est la dispersion des valeurs individuelles ?
u sur la moyenne = s / racine(n)	Quelle incertitude-type attribue-t-on à la moyenne du fait de cette dispersion ?

Pourquoi ddof=1 ?

Pour une série expérimentale utilisée comme échantillon, **np.std(mesures, ddof=1)** calcule l'écart-type expérimental. Sans cet argument, NumPy utilise par défaut une autre convention.

Intervalle à 95 %

Le script construit l'intervalle approximatif **moyenne ± 2 u_moyenne** dans l'hypothèse d'un comportement normal. Pour une petite série, le facteur exact dépend du niveau de confiance et du nombre de degrés de liberté ; le facteur 2 reste ici une approximation pédagogique.

Écart normalisé

```
En = abs(x1 - x2) / sqrt(u1**2 + u2**2)

if En <= 2:
    print("Valeurs compatibles avec le critère choisi")
```

Le critère **En <= 2** conduit à parler de valeurs compatibles au regard des incertitudes déclarées. Il ne prouve pas que les valeurs sont identiques. Si **En > 2**, il faut rechercher une cause d'incompatibilité.

2. Effectuer une régression linéaire

Une régression affine cherche les paramètres **a** et **b** du modèle $y = ax + b$ qui décrivent au mieux les données selon la méthode utilisée.

Commande principale

```
a, b = np.polyfit(x, y, 1)
```

Le nombre **1** indique le degré du polynôme : il impose ici une droite. La fonction renvoie d'abord la pente, puis l'ordonnée à l'origine.

Tracer mesures et modèle

```
y_modele = a * x + b  
  
plt.scatter(x, y, label="Mesures")  
plt.plot(x, y_modele, label="Régression")
```

Exemple d'étalonnage

Résultat du script 03	Valeur approchée	Sens
a	592	Sensibilité : variation d'absorbance par mol/L
b	-0,00025	Valeur prévue lorsque la concentration vaut zéro

Questions à poser

- Le modèle affine est-il physiquement pertinent ?
- Le domaine étudié est-il celui dans lequel la loi est attendue ?
- L'ordonnée à l'origine est-elle cohérente avec le blanc et le protocole ?
- Les unités de la pente ont-elles un sens ?
- Un point exerce-t-il une influence excessive sur la droite ?

La valeur de R^2 ne suffit pas à valider une régression. Une droite peut présenter un R^2 élevé tout en étant inadaptée, notamment si les résidus sont structurés ou si le domaine de linéarité est dépassé.

3. Valider une régression

Le script 04 associe deux lectures complémentaires : le graphe des mesures avec leurs barres d'incertitude et le graphe des résidus normalisés.

Barres d'incertitude

```
axes[0].errorbar(  
    x, y,  
    yerr=u_y,  
    fmt="o", capsize=4  
)
```

Chaque barre représente ici l'incertitude-type portée par l'ordonnée. Le graphique montre si l'écart entre la mesure et la droite est grand ou petit à l'échelle de cette incertitude.

Résidus normalisés

```
residus = y - y_modele  
residus_normalises = residus / u_y  
  
axes[1].scatter(x, residus_normalises)  
axes[1].axhline(2, color="red", linestyle="--")  
axes[1].axhline(-2, color="red", linestyle="--")
```

Lire le graphe

Observation	Interprétation possible
Résidus dispersés sans structure autour de zéro	Le modèle affine reste plausible dans le domaine étudié.
Courbure visible dans les résidus	Le modèle affine ou le domaine de linéarité est probablement inadapté.
Un résidu au-delà de ± 2	Examiner la mesure, son incertitude et les conditions expérimentales.
Tous les résidus du même signe sur une zone	Rechercher un biais local ou une structure ignorée par le modèle.

La limite ± 2 est un critère de lecture lié aux incertitudes déclarées. Un point extérieur n'est pas automatiquement « faux » : il signale une incompatibilité à analyser.

Bon réflexe

Toujours conserver le graphe original avec les unités. Un graphe de résidus seul ne permet pas de comprendre le domaine, la répartition des abscisses ni la grandeur physique étudiée.

4. Tirages normaux et Monte-Carlo

Un tirage aléatoire représente une valeur possible d'une grandeur dont la valeur centrale et l'incertitude-type sont connues.

Créer une population simulée

```
tirages = np.random.normal(
    valeur_centrale,
    incertitude_type,
    nombre_de_tirages
)
```

Le deuxième argument est l'écart-type de la loi normale, donc l'incertitude-type utilisée dans le modèle. Il ne s'agit ni d'une tolérance complète ni d'une incertitude élargie.

Propagation d'une grandeur composée

Pour un calcul de titrage, le script 06 tire séparément une concentration de titrant, un volume équivalent et un volume prélevé, puis recalcule la concentration recherchée.

```
c_sim = np.random.normal(c, u_c, N)
veq_sim = np.random.normal(veq, u_veq, N)
vp_sim = np.random.normal(vp, u_vp, N)

resultats = dilution * c_sim * veq_sim / vp_sim
valeur = np.mean(resultats)
u_valeur = np.std(resultats, ddof=1)
```

Étape	Décision à documenter
1. Entrées	Valeur centrale et incertitude-type de chaque grandeur
2. Distributions	Loi normale ou autre loi justifiée
3. Modèle	Relation de calcul identique à celle du compte rendu
4. Tirages	Nombre suffisamment grand et, pour apprendre, graine fixée
5. Sortie	Moyenne, écart-type, histogramme et unité

Une simulation de 10 000 tirages ne remplace pas 10 000 expériences. Elle propage les distributions et les hypothèses choisies à travers un modèle mathématique.

5. Monte-Carlo sur une régression

Le script 07 simule la variation des absorbances. Une nouvelle droite est ajustée à chaque série simulée ; la dispersion des pentes et des ordonnées à l'origine fournit une estimation de leurs incertitudes-types.

```
for i in range(nombre_de_simulations):
    y_sim = np.random.normal(y, u_y)
    a_sim, b_sim = np.polyfit(x, y_sim, 1)
    pentes.append(a_sim)
    origines.append(b_sim)

u_pente = np.std(pentes, ddof=1)
```

Réutilisation dans les TP

TP	Exploitation	Scripts
TP1 - Vinaigre	Type A, comparaison des méthodes, propagation du titrage	01, 02, 06
TP2 - Spectrophotométrie	Étalonnage, validation et incertitude sur la pente	03, 04, 07
TP6 - Complexe FeSCN	Étalonnage et propagation jusqu'à la concentration	03, 04, 07
TP7 - Partage	Propagation vers la constante ou le rendement	06 à adapter
TP8, TP9, TP14	Type A, écart normalisé et propagation d'un titrage	01, 02, 06

Erreurs fréquentes

- Employer **np.std** sans préciser la convention souhaitée.
- Confondre écart-type des mesures et incertitude-type sur leur moyenne.
- Déclarer deux valeurs identiques lorsque En est inférieur à 2.
- Valider une droite uniquement avec R^2 .
- Utiliser une tolérance comme écart-type sans conversion justifiée.
- Simuler des entrées sans indiquer leurs distributions ni leurs unités.
- Arrondir les valeurs avant la fin des calculs.

Validation rapide

Je sais : calculer moyenne, écart-type et u sur la moyenne ; interpréter En ; utiliser **polyfit** ; tracer barres et résidus normalisés ; créer des tirages avec **random.normal** ; propager un modèle ; estimer l'incertitude d'une pente par Monte-Carlo.

À retenir

Les statistiques quantifient la variabilité ; la validation confronte données, modèle et incertitudes ; Monte-Carlo propage explicitement les hypothèses retenues.